Python Naming Conventions Cheat Sheet

Python Class (10-152-117)
Reference: PEP 8 - Style Guide for Python Code

The Python quick rule

Use `snake_case` for most names, `PascalCase` for classes, and `UPPER_SNAKE_CASE` for constants. Names should describe the value, behavior, or responsibility they represent.

Use in Python	Preferred convention	Example
Variables	`snake_case`	`total_cost`
Functions and methods	`snake_case`	`calculate_total()`
Modules and packages	short `snake_case`	`grade_tools.py`
Classes	`PascalCase`	`GradeCalculator`
Exceptions	`PascalCase`	`InvalidGradeError`
Constants	`UPPER_SNAKE_CASE`	`MAX_ATTEMPTS = 3`

Common naming conventions

Convention	Example	Where you may see it	Python status
lowercase	`score`	Simple names	Common
snake_case	`student_score`	Python variables, functions, methods, modules	**PEP 8 preferred**
PascalCase (CapitalCase)	`StudentScore`	Classes and exceptions	**PEP 8 preferred**
camelCase	`studentScore`	JavaScript, Java, C#	Avoid for Python names
UPPER_SNAKE_CASE (SCREAMING_SNAKE_CASE)	`MAX_SCORE`	Constants and configuration	**PEP 8 preferred**
kebab-case (train-case)	`student-score`	URLs, CSS classes, command-line options	Not valid for Python identifiers
dot.case	`student.score`	Module paths and attribute access	Not a single identifier
Title Case	`Student Score`	Headings and	Not a Python

Convention	Example	Where you may see it	Python status
		human-facing labels	identifier
flatcase	`studentscore`	Occasional file names or labels	Rare; less readable
UPPERCASE flatcase	`STUDENTSCORE`	Acronyms or legacy systems	Rare; avoid for new Python names

Python-specific underscore conventions

Pattern	Example	Meaning
Leading underscore	`_cached_result`	Conventionally internal or non-public. `from module import *` does not import it.
Trailing underscore	`class_`	Avoids a conflict with a Python keyword or built-in name.
Double leading underscore	`__balance`	Python applies name mangling inside a class to help avoid accidental subclass conflicts.
Double leading and trailing underscores (dunder)	`__init__`, `__len__`	Special names defined by Python. Do not invent new dunder names.

Identifier rules

- Start with a letter or underscore, followed by letters, digits, or underscores.
- Do not start with a digit: `2nd_attempt` is invalid.
- Do not use spaces or hyphens: `student score` and `student-score` are invalid.
- Python is case-sensitive: `score`, `Score`, and `SCORE` are different names.
- Do not reuse Python keywords such as `class`, `for`, `if`, or `return`.
- Prefer descriptive names: `average_score` is clearer than `avg` when space allows.

PEP 8 reminders

- Choose names that reveal intent: `is_complete` is clearer than `flag`.
- Use verbs for actions: `save_file()`, `calculate_total()`, and `display_menu()`.
- Use nouns for values and objects: `student_name`, `shopping_cart`, and `report`.
- Use boolean names that read like questions: `is_valid`, `has_access`, and `can_submit`.
- Be consistent. A project using `student_name` should not also use `studentName`.

Quick practice

Rewrite these names using Python conventions:

Less clear or non-Python style	Python-style revision
`studentName`	`student_name`
`CalculateTotal()`	`calculate_total()`
`gradecalculator`	`GradeCalculator`
`maxRetries`	`MAX_RETRIES`
`class`	`class_`

Key takeaway: consistent, descriptive names make code easier to read, debug, and maintain.